Spark & YARN Configuration Best Practices

Prepared for : Allegiance

Version	Created by	Reviewed by	Date	Remarks
1.0	Kasim Sheikh	Karthik Hegde H A	May 04,2026	Knowledge Base

Table of Contents

Spark & YARN Configuration Best Practices....1
Table of Contents....2
Document Purpose....4
1. Workload Context & Critical Constraints....4
1.1 Raw Data Characteristics....4
1.2 Job Pattern....5
1.3 SLA Risk Summary....5
2. The Core Issue: GZIP is Non-Splittable....6
2.1. Chunk the Files Upstream (Best Practice / Zero SLA Risk)....6
2.2. Change Compression Codec to bzip2 (Excellent / Low SLA Risk)....6
2.3. Send Uncompressed CSVs (Good / Medium SLA Risk)....6
2.4. Post-Read Repartitioning (Last Resort Mitigation / High SLA Risk)....7
3. OS & Filesystem Configuration....7
3.1 Kernel Tuning (sysctl.conf)....7
3.2 Transparent Huge Pages (THP) — Disable....8
3.3 NVMe Scratch Filesystem — XFS....8
3.4 External Shuffle Service JAR....9
4. YARN Configuration....9
4.1 yarn-env.sh....9
4.2 yarn-site.xml — Key Settings....10
5. Spark Configuration — spark-defaults.conf....10
5.1 Core & Memory Sizing....10
5.2 Job Reliability & Storage....11
5.3 Compression Codecs....11
5.4 Serialization....11
5.5 S3A Committer & Fast Upload....12
5.6 SQL Performance....13
5.7 YARN JAR Caching....13
6. S3A & Ceph RGW Tuning....13
6.1 Ceph RGW Endpoint & Authentication....13
6.2 Kerberos Authentication (HIPAA)....14
6.3 S3A Connection Pool & Parallelism....14
6.4 Read & Write Optimization....15
7. Adaptive Query Execution & Partition Strategy....15

7.1 AQE Configuration..... 15
7.2 Repartition Code Template..... 16
8. HIPAA Compliance Architecture & Guidelines.....17
8.1. Transmission Security (Data in Transit)..... 17
8.2. Data at Rest Encryption..... 18
8.3 Web UI Security & Access Control (ACLs) - javax servlet filters Requirement..... 19
9. Configuration Reference (3 upstream options).....20
9.1 S3A Read & Partition Tuning Options by Data Flow.....20

Document Purpose

This document provides authoritative Spark 3.5.3 and YARN 3.4.1 configuration guidance for the development cluster. incorporates actual DEV environment table load data shared by the client (csv_file_sizes.xlsx)

Workload (actual): 800 tables mapped ·

- 359 with DEV data
- 12,214 files
- ~1.52 TB compressed CSV
- CSV ➡️ Parquet ➡️ Snowflake

1. Workload Context & Critical Constraints

Config parameters in this document are driven by the actual client DEV environment load data.

1.1 Raw Data Characteristics

Property	Detail
Format	CSV compressed with GZIP (.csv.gz)
Total Tables in Mapping	800 tables total in the mapping definition
Tables with DEV Data	359 tables (ESTIMATED status with actual file content having CSV size > 0)
Tables Empty / Placeholder	257 tables (size = 0 KB and ESTIMATED status — schema-only DEV placeholders)
Tables with NO_FILES & NO_FILEFORM	184 tables (not yet populated in DEV environment)
Total Files (DEV batch)	~12214 files (2 × 5,800 multi-file tables + 357single-file tables)
Batch Volume (DEV, compressed)	1.52 TB compressed CSV / 2.17 TB source — NOT the 4 TB originally assumed (DEV env is a subset of full PROD load)
Multi-file Tables	2 tables only, each with exactly 5,800 files: Table A (527.95 GB CSV, 293 cols, ~97 MB/file avg) and Table B (113.10 GB CSV, 94 cols, ~21 MB/file avg)
Single-file Table Range	1 KB – 145.57 GB as a single .csv.gz file (extreme variance)

Property	Detail
Widest Tables	Max 384 cols (one table); 14 tables have >100 cols. Wide schemas impact sort memory.
File-to-Table Mapping	Excel/DB mapping file consumed by Spark job at runtime
S3 Layout	s3a://ingest/csv/<date>/ — flat directory per batch date
Output	Parquet (ZSTD) → s3a://output/parquet/<table>/
Pipeline	Airflow DAG → spark-submit per table → Ceph → Parquet → Snowflake COPY INTO

⚠ CRITICAL — GZIP NON-SPLITTABILITY:
GZIP IS NOT SPLITTABLE. A .csv.gz file cannot be split across Spark tasks regardless of size. A 145 GB single .gz file becomes exactly 1 Spark partition = 1 task = 1 executor core reading the entire file sequentially. This is the dominant constraint for the large single-file tables in this dataset. All 8 executor cores sit idle during the read stage for these tables until a repartition() call redistributes data

1.2 Job Pattern

Each Spark job corresponds to one target table. Airflow orchestrates job submission. The cluster supports 1 concurrent Spark job with the current 3-node configuration.

- Job ordering: schedule the 2 multi-file tables and the 10 ">=20 GB" single-file tables FIRST. They consume the most SLA time and must not be deferred.
- The 149 placeholder tables (<1 MB) will complete in seconds — group these at the end or skip with a size check in the DAG.
- Files are deleted from Ceph after successful Snowflake COPY INTO confirmation.

1.3 SLA Risk Summary

Metric	Value
Total jobs / day (DEV)	35-40 jobs
Volume domination	Top 12 tables (2 multi-file + 10 >20 GB) = 1,234.15 GB = 79.29% of total 1.52 TB
Small job majority	347 tables < 20 GB each; most complete in 5 min to 10 min — bulk of jobs are fast
SLA dominant jobs	Multi-file Table A (527 GB) is the single largest job. At 1 TB/hr = ~32 min. Must run first.
SLA window	4 hours (240 min)

Metric	Value
Critical path estimate	Multi-file A (~32 min) + Multi-file B (~7 min) + top 10 single-file tables (~120 min) = ~159 min for top 12 tables alone. Leaves ~81 min for 347 smaller jobs.
SLA feasibility	Feasible if large tables run sequentially first. Zero buffer at 40+ large tables. Phase 1 (+1 node → 2 concurrent jobs) strongly recommended.
Recommendation	Schedule large tables first in Airflow. Add size-check gate to skip effectively-empty placeholder tables.

2. The Core Issue: GZIP is Non-Splittable

The single most critical technical constraint in this workload is that GZIP (.gz) is not a splittable compression format. Configuration settings like spark.sql.files.maxPartitionBytes have absolutely no effect on non-splittable formats like GZIP.

2.1. Chunk the Files Upstream (Best Practice / Zero SLA Risk)

The Solution: The source system generating the data splits the massive tables into smaller chunks (e.g., 100 MB to 200 MB each) before compressing them into .csv.gz and uploading them.
Why it's the best: Aligns perfectly with Spark's architecture. Spark naturally assigns one core to each small .gz file.
Trade-off: Requires coordination and engineering effort from upstream data providers.

2.2. Change Compression Codec to bzip2 (Excellent / Low SLA Risk)

The Solution: Change the compression format from GZIP (.gz) to bzip2 (.bz2).
Why it works: bzip2 contains internal block markers. Spark natively recognizes bzip2 as a splittable format and slices it into 128 MB partitions automatically.
Trade-off: bzip2 is slightly slower to compress/decompress, but parallel processing vastly outweighs this.

2.3. Send Uncompressed CSVs (Good / Medium SLA Risk)

The Solution: The upstream system sends massive files as raw, uncompressed .csv files.

Why it works: Raw text files are natively splittable in Spark.

Trade-off: Drastically increases storage footprint on Ceph and network transfer time (I/O).

2.4. Post-Read Repartitioning (Last Resort Mitigation / High SLA Risk)

The Solution: Developers must use df.repartition() immediately after reading the file.
Why it works: Forces Spark to shuffle data to all available task slots for downstream transformations.
Trade-off: Does not fix the initial read bottleneck, risking OOM crashes before repartitioning occurs.

3. OS & Filesystem Configuration

Applied via Ansible pre-tasks on all YARN NodeManager VMs (RHEL 9.4).

3.1 Kernel Tuning (sysctl.conf)

Parameter	Value	Rationale
vm.swappiness	1	Forces RHEL to use physical RAM; prevents JVM heap swapping that causes GC pauses.
vm.dirty_ratio	40	Max % of memory holding dirty pages before forced fsync. Lets Spark buffer shuffle writes.
vm.dirty_background_ratio	10	Background writeback threshold. Staggers writes smoothly with dirty_ratio=40.
net.core.rmem_max	134217728	128 MB socket receive buffer — required for sustained 10G throughput during shuffle fetch.
net.core.wmem_max	134217728	128 MB socket send buffer maximum.
net.ipv4.tcp_rmem	4096 87380 134217728	TCP receive buffer min/default/max.
net.ipv4.tcp_wmem	4096 65536 134217728	TCP send buffer min/default/max.
net.core.somaxconn	4096	Maximum socket backlog — prevents NodeManager drops under high concurrency.

Parameter	Value	Rationale
kernel.pid_max	4194304	Prevents PID exhaustion for JVM and Spark task threads.

3.2 Transparent Huge Pages (THP) — Disable

Mandatory Requirement: Transparent HugePages (THP) must be DISABLED. Add these to /etc/rc.d/rc.local:

```
echo never > /sys/kernel/mm/transparent_hugepage/enabled
echo never > /sys/kernel/mm/transparent_hugepage/defrag
```

3.3 NVMe Scratch Filesystem — XFS

Format all 4 NVMe scratch drives per node (drives 4–7) as XFS using the following exact mount options.

1. Format the Drives Run the mkfs.xfs command for each of the four NVMe drives (/dev/nvme3n1, /dev/nvme4n1, /dev/nvme5n1, and /dev/nvme6n1):

```shell
mkfs.xfs -f -d su=256k,sw=1 /dev/nvme3n1
mkfs.xfs -f -d su=256k,sw=1 /dev/nvme4n1
mkfs.xfs -f -d su=256k,sw=1 /dev/nvme5n1
mkfs.xfs -f -d su=256k,sw=1 /dev/nvme6n1
```

2. Update /etc/fstab Add the following lines to your /etc/fstab file to ensure the drives mount correctly on boot with the required performance flags:

```shell
# NVMe Scratch Drives
/dev/nvme3n1 /mnt/nvme-scratch/d0 xfs defaults,noatime,nodiratime,discard,allocsize=64m 0 0
/dev/nvme4n1 /mnt/nvme-scratch/d1 xfs defaults,noatime,nodiratime,discard,allocsize=64m 0 0
/dev/nvme5n1 /mnt/nvme-scratch/d2 xfs defaults,noatime,nodiratime,discard,allocsize=64m 0 0
/dev/nvme6n1 /mnt/nvme-scratch/d3 xfs defaults,noatime,nodiratime,discard,allocsize=64m 0 0
```

Mount Options Rationale

- **noatime / nodiratime:** Eliminates metadata write-overhead on every file read. This is

absolutely critical for shuffle-heavy Spark workloads.

- **discard:** Enables NVMe TRIM to preserve drive health and maintain sustained write performance over time.
- **allocsize=64m:** Forces XFS to pre-allocate larger file extents, significantly reducing disk fragmentation when Spark writes large shuffle files.

3.4 External Shuffle Service JAR

Mandatory before starting YARN NodeManager. Enables the External Shuffle Service (ESS) so executor containers can terminate without losing shuffle output.

1. Copy the Shuffle JAR Copy the Spark YARN shuffle JAR into your Hadoop YARN library directory:

```shell
cp $SPARK_HOME/yarn/spark-3.5.3-yarn-shuffle.jar $HADOOP_HOME/share/hadoop/yarn/lib/
```

2. Verify the Classpath Ensure the JAR is now successfully recognized in the Hadoop classpath:

Bash

```shell
hadoop classpath | grep shuffle
```

> ⚠ **Deployment Note — Hadoop Version Mismatch**
> Spark 3.5.x ships with Hadoop 3.3.x. This cluster uses standalone Hadoop 3.4.1. Ensure HADOOP_HOME points to 3.4.1 and spark.yarn.populateHadoopClasspath = true is set so Spark uses the correct S3A and YARN libraries.

4. YARN Configuration

4.1 yarn-env.sh

Parameter	Value	Rationale
YARN_HEAPSIZE	2048	Raises NodeManager JVM heap to 2048 MB to serve heavy shuffles.
YARN_RESOURCEMANAGER_HEAPSIZE	1024	Sufficient for RM to schedule sequential jobs.

4.2 yarn-site.xml — Key Settings

Parameter	Value	Rationale
yarn.nodemanager.vmem-check-enabled	false	Prevents YARN killing executors for normal JVM virtual memory spikes.
yarn.nodemanager.pmem-check-enabled	false	Spark manages its own physical memory.
yarn.nodemanager.local-dirs	/mnt/nvme-scratch/d0/yarn,...	Round-robins YARN temp across all 4 NVMe drives.
yarn.nodemanager.aux-services	spark_shuffle	Enables External Shuffle Service.
yarn.log-aggregation-enable	true	Ships container logs to Ceph for History Server.
yarn.scheduler.maximum-allocation-mb	32768	Cap per-container at 32 GB. Matches 28g heap + 4g overhead.
yarn.scheduler.maximum-allocation-vcores	8	Matches spark.executor.cores = 8.

5. Spark Configuration — spark-defaults.conf

5.1 Core & Memory Sizing

Parameter	Value	Rationale
spark.executor.cores	8	Optimal for S3A/Ceph RGW parallel reads.
spark.executor.memory	28g	28 GB heap per executor.
spark.driver.cores	4	Coordinates DAG scheduling.
spark.driver.memory	8g	Sufficient for partition tracking and DAG plan.

5.2 Job Reliability & Storage

Parameter	Value	Rationale
spark.yarn.maxAppAttempts	1	Fail-fast. Airflow handles retry logic.YARN retries cause duplicate side-effects. Can set higher if airflow will not retry
spark.local.dir	/mnt/nvme-scratch/...	Round-robins shuffle across 4 NVMe drives.
spark.shuffle.file.buffer	1m	In-memory buffer per shuffle map file.
spark.reducer.maxSizeInFlight	96m	Data fetched per reduce task simultaneously.
spark.network.timeout	300s	Extended for S3A ops during peak Ceph load. Default 120s insufficient for large shuffles.
spark.sql.broadcastTimeout	300s	Broadcast join timeout. The table mapping file broadcast may be slow on first load.
spark.rpc.askTimeout	300s	Driver-executor RPC timeout. Keep consistent with network.timeout.

5.3 Compression Codecs

Parameter	Value	Rationale
spark.io.compression.codec	snappy	Snappy for shuffle: 2–3× compression at very low CPU cost. Better ratio than LZ4 (1.5–2×). CPU-constrained cluster favours Snappy.
spark.sql.parquet.compression.codec	zstd	ZSTD for Parquet output: ~5× compression on columnar typed data. Reduces Ceph OSD footprint to ~0.3 TB (from 1.52 TB compressed CSV).

5.4 Serialization

Parameter	Value	Rationale
spark.serializer	org.apache.spark.serializer.KryoSerializer	Kryo is 2–5× faster than Java default serializer for shuffle and broadcast. Required for all production Spark ETL.

spark.kryoserializer.buffer.max	512m	Prevents buffer overflow when broadcasting the 5800-row mapping file or wide-schema metadata to all executors.

5.5 S3A Committer & Fast Upload

Parameter	Value	Rationale
spark.hadoop.fs.s3a.committer.name	magic	Bypasses slow S3 rename (copy+delete). Magic committer uses multipart upload directly. Saves 20–30% job time.
spark.hadoop.fs.s3a.committer.magic.enabled	true	Activates the magic committer explicitly.
spark.hadoop.fs.s3a.committer.staging.conflict-mode	replace	Idempotent — safe for Airflow-retried jobs.
spark.sql.sources.commitProtocolClass	org.apache.spark.internal.io.cloud.PathOutputCommitProtocol	Wires magic committer into Spark SQL DataFrameWriter.save().
spark.sql.parquet.output.committer.class	org.apache.hadoop.mapreduce.lib.output.BindingPathOutputCommitter	Parquet has historically implemented its own custom output committer, completely bypassing Hadoop's standard MapReduce committers. This property binds the Parquet writer specifically to the Hadoop cloud committer architecture so it doesn't go rogue
spark.hadoop.fs.s3a.fast.upload	true	Streams Parquet output to Ceph in background while tasks continue processing.
spark.hadoop.fs.s3a.fast.upload.buffer	disk	NVMe scratch buffers upload chunks — keeps 28g heap free for shuffle and aggregation.
spark.hadoop.fs.s3a.multipart.size	128M	128 MB multipart chunk — optimal for 10G bond.
spark.hadoop.fs.s3a.multipart.threshold	128M	Objects above this threshold use multipart upload.

5.6 SQL Performance

Parameter	Value	Rationale
spark.sql.autoBroadcastJoinThreshold	52428800	50 MB. Reference/lookup tables smaller than 50 MB are automatically broadcast, eliminating a shuffle stage.
spark.sql.inMemoryColumnarStorage.compressed	true	Compress in-memory cached DataFrames if any jobs use .cache().
spark.sql.inMemoryColumnarStorage.batchSize	10000	12 tables have >100 cols, including one with 384 cols. Smaller batch size reduces per-batch memory for wide schemas — prevents GC pressure on columnar operations.
spark.sql.join.preferSortMergeJoin	false	Allows broadcast hash join when autoBroadcastJoinThreshold qualifies.

5.7 YARN JAR Caching

Parameter	Value	Rationale
spark.yarn.archive	s3a://spark-binaries/spark-3.5.3-jars.zip	Pre-upload Spark JAR bundle once. Saves time per job to load the spark binary jars.
spark.yarn.populateHadoopClasspath	true	Forces Spark to use standalone Hadoop 3.4.1 libraries for optimized S3A connector.

6. S3A & Ceph RGW Tuning

6.1 Ceph RGW Endpoint & Authentication

Parameter	Value	Rationale
spark.hadoop.fs.s3a.endpoint	https://<CEPH_RGW_VIP>:443	CRITICAL — S3A defaults to AWS S3. Replace with Ceph RGW VIP or DNS name.
spark.hadoop.fs.s3a.path.style.access	true	CRITICAL — Ceph requires path-style access. Without this all S3A requests fail with DNS errors.
spark.hadoop.fs.s3a.access.key	<inject via Ansible Vault>	Store in Ansible Vault. Never commit plaintext.
spark.hadoop.fs.s3a.secret.key	<inject via Ansible Vault>	Store in Ansible Vault.

spark.hadoop.fs.s3a.aws.credentials.provider	org.apache.hadoop.fs.s3a.SimpleAWSCredentialsProvider	Static credentials provider for Ceph RGW.

6.2 Kerberos Authentication (HIPAA)

Parameter	Value	Rationale
spark.kerberos.principal	spark-etl@<YOUR_REALM>	Dedicated service principal. Replace <YOUR_REALM> with actual KDC realm. Do not use hdfs or yarn principal.
spark.kerberos.keytab	/etc/security/keytabs/spark-etl.keytab	Keytab deployed by Ansible with owner=spark, mode=0400. HIPAA mandatory.
spark.yarn.kerberos.relogin.period	1m	Refreshes Kerberos token every minute. Prevents Credentials Expired errors on long-running jobs (145 GB table can run 30+ min).
spark.redaction.regex	(?i)secret|password|token|keytab|principal|credential|key	HIPAA — redacts credentials from Spark UI, event logs, and driver logs.
spark.redaction.string	********	Replacement string for redacted values.

6.3 S3A Connection Pool & Parallelism

Parameter	Value	Rationale
spark.hadoop.fs.s3a.threads.max	20	Max parallel upload threads per S3A client. Default 10 is insufficient for 8-core executor profile.
spark.hadoop.fs.s3a.connection.maximum	20	Max HTTP connections per S3A client. Must be >= threads.max.
spark.hadoop.fs.s3a.socket.timeout	60000	60s socket timeout. Fail fast on stalled connections — retry via task.maxFailures.
spark.hadoop.fs.s3a.connection.timeout	5000	5s TCP connection establishment timeout. On LAN, should be near-instant.
spark.hadoop.fs.s3a.attempts.maximum	5	Request retries before failing the task.
spark.hadoop.fs.s3a.retry.limit	7	Retries for throttling/503 errors. Higher than attempts.maximum.

6.4 Read & Write Optimization

Parameter	Value	Rationale
spark.hadoop.fs.s3a.paging.maximum	5000	Max objects per S3 LIST. The 5800-file table will LIST in 2 pages (5000 + 800) — normal behaviour.
spark.hadoop.fs.s3a.block.size	134217728	128 MB upload block size consistent with multipart.size.
spark.hadoop.fs.s3a.fast.upload.active.blocks	4	4 × 128 MB = 512 MB NVMe scratch per executor for active uploads.

⚠ **Ceph-Side RGW Tuning (Ceph Admin Action Required)**

Ask the Ceph admin to apply: ceph config set client.rgw rgw_thread_pool_size 512 and rgw_max_concurrent_requests 1024. Default thread pool becomes a bottleneck when 3 executors × 8 cores simultaneously issue range reads against 3 RGW instances — especially during the 5800-partition multi-file table jobs.

7. Adaptive Query Execution & Partition Strategy

7.1 AQE Configuration

Parameter	Value	Rationale
spark.sql.adaptive.enabled	true	Enables AQE globally. Spark 3.2+ default — verify not overridden.
spark.sql.adaptive.coalescePartitions.enabled	true	Critical for the 166 tiny-table jobs. AQE merges 4096 nearly-empty shuffle partitions to 1–2 real ones automatically.
spark.sql.adaptive.skewJoin.enabled	true	Essential for this dataset. Multi-file Table A has 293 columns — skew on high-cardinality join keys is likely.
spark.sql.adaptive.skewJoin.skewedPartitionFactor	5	A partition is skewed if 5× the median. Reasonable for this mixed-size workload.
spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes	268435456	256 MB minimum before skew detection triggers. Prevents false positives on small tables.
spark.sql.shuffle.partitions	4096	Rationale: Multi-file Table A (527 GB compressed, ~737 GB uncompressed) needs ~5896 shuffle partitions at 128 MB target. 4096 is a practical balance — AQE handles small tables by coalescing down. Using 2000 left the largest table under-partitioned, creating 369 MB shuffle chunks and reducing parallelism.

7.2 Repartition Code Template

⚠ Developer Action — Repartition After Read (Mandatory for Large Tables)

For any table where the CSV file(s) decompress to >1 GB, the job code MUST call repartition() after spark.read.csv(). Without this, all processing happens in a single task on a single core. Use the formula: repartition_count = max(24, ceil(compressed_csv_mb / 50)). This formula targets ~50 MB compressed per partition, giving 2–3 partitions per executor core. The Airflow DAG should pass the expected file size as a job parameter so the Spark job can auto-calculate the repartition count at runtime.

Embed these patterns in the ETL job code. The table_size_mb value should be passed from the Airflow DAG via --conf or job argument:

Single-file tables (All tables except the two 5,800-file tables)

```python
# compressed_mb is injected by Airflow at runtime
compressed_mb = <table_csv_size_mb>

# 50 MB compressed → ~70-100 MB uncompressed per partition
target_partitions = max(24, int(compressed_mb / 50))

df = spark.read.option("header", "true").csv("s3a://ingest/csv/<date>/<table>/*.csv.gz")

# Spread across all 24 task slots
df = df.repartition(target_partitions)

# Example: 145 GB table → max(24, 145000/50) = 2900 partitions
```

---

8. HIPAA Compliance Architecture & Guidelines

8.1. Transmission Security (Data in Transit)

HIPAA Reference: 45 CFR § 164.312(e)(1)

HIPAA requires encryption for all ePHI traversing the network. In a distributed Spark environment, this involves both external communication (to Ceph RGW) and internal communication (between Driver, Executors, and YARN).

- **External TLS/SSL (Ceph S3A):** All API calls between Spark and Ceph RGW must use HTTPS.
- **RPC Authentication:** Spark processes (RPC channels) must be authenticated. Deploying on YARN allows Spark to automatically generate and securely distribute a unique shared secret per application, but authentication must be explicitly enabled.
- **Internal RPC & Shuffle Encryption:** During wide transformations, ePHI is shuffled across the internal network. AES-GCM encryption must be enabled for these RPC connections. Version 2 of the authentication engine is required, as Version 1 omits applying a Key Derivation Function (KDF).
- **Global SSL Namespace:** SSL must be globally enabled to ensure the Web UI and History Server utilize certificates.

Required Configuration:

Parameter	Value	Rationale
spark.hadoop.fs.s3a.connection.ssl.enabled	true	CRITICAL: Forces HTTPS for all data transmission between Spark and Ceph RGW.
spark.authenticate	true	Mandates authentication for internal RPC connections.
spark.network.crypto.enabled	true	Enables AES-based encryption for all internal Spark RPC and shuffle data transfers.
spark.network.crypto.authEngineVersion	2	Enforces the secure Version 2 protocol (uniformly distributed derived session key).
spark.network.crypto.cipher	AES/GCM/NoPadding	Upgrades the cipher to an authenticated encryption mode (GCM).

spark.ssl.enabled	true	Globally enables SSL for the Web UI and History Server.
spark.ssl.rpc.enabled	true	Explicitly enables SSL for RPC connections (Netty), taking precedence over AES if both are defined.

8.2. Data at Rest Encryption

HIPAA Reference: 45 CFR § 164.312(a)(2)(iv)

HIPAA mandates the protection of stored ePHI to prevent unauthorized access. In Spark, data at rest includes source data, output data, and the temporary spill/shuffle data written to local disks.

- **Local I/O Storage Encryption:** Spark must be configured to natively encrypt temporary data written to local disks (shuffle files, spills, and cache blocks).
- **Physical Disk Encryption:** The 4 NVMe scratch drives per node (/dev/nvme3n1 through /dev/nvme6n1) should additionally be encrypted at the OS level using LUKS (Linux Unified Key Setup) before being formatted with XFS.
- **Ceph Object Storage:** Ceph RGW must enforce Server-Side Encryption (SSE-KMS or SSE-S3) for the `s3a://ingest/` and `s3a://output/` buckets.

Required Configuration:

Parameter	Value	Rationale
spark.io.encryption.enabled	true	Encrypts all local disk I/O (shuffle files and spills) before writing to the NVMe drives.
spark.io.encryption.keySizeBits	256	Enforces the maximum 256-bit encryption key size for local I/O.
spark.io.encryption.keygen.algorithm	HmacSHA256	Uses a strong cryptographic hashing algorithm for generating the IO encryption key.

8.3 Web UI Security & Access Control (ACLs) - javax servlet filters Requirement

The Spark Web UI and History Server display job metadata. ACLs must be enforced so only authorized administrators can access application details. **Enabling authentication for the Web UIs is done using javax servlet filters.** Spark does not provide any built-in authentication filters.

> ⚠ **YARN Mode Exception**
> When running Spark on YARN, the spark.ui.filters property is often automatically overridden by the system to use the org.apache.hadoop.yarn.server.webproxy.amfilter.AmIpFilter. This filter ensures that only the YARN Web Proxy can access the Application Master's UI.
>
> But this is only for Running jobs and does not apply to history server, you need a **javax servlet filter** for the Spark History Server

1. You must deploy a filter implementing your authentication method (e.g., LDAP) using javax servlet filters.
2. Place the filter JAR in the Spark classpath.
3. Configure spark.ui.filters to point to your filter's fully qualified class name.
4. Only then will spark.acls.enable = true successfully restrict authenticated users.
5. Refer : https://downloads.apache.org/spark/docs/3.5.3/security.html

Required Configuration:

Parameter	Value	Rationale
spark.acls.enable	true	Enforces view/modify permissions on the active Spark Web UI.
spark.history.ui.acls.enable	true	Enforces ACLs on the Spark History Server, overriding any application-level ACL settings.
spark.admin.acls	<admin_users>	Comma-separated list of highly privileged users allowed to modify/kill jobs.
spark.ui.filters	com.yourcompany.security.CustomAuthFilter	Prerequisite Filter Class
spark.ui.xXssProtection	1; mode=block	Enables browser XSS filtering and blocks page rendering if

		an attack is detected.
spark.ui.xContentTypeOptions.enabled	true	Prevents the browser from MIME-sniffing the response payload (sets nosniff).
spark.ui.strictTransportSecurity	max-age=31536000; includeSubDomains	Enforces HTTP Strict Transport Security (HSTS) for 1 year, requiring SSL for all UI connections.

9. Configuration Reference (3 upstream options)

9.1 S3A Read & Partition Tuning Options by Data Flow

Choose the configuration column below that matches their exact upstream data flow.

Configuration Property	Value (Pre-split GZIP)	Value (BZIP2)	Value (Raw CSV)	Configuration Description & Rationale
spark.executor.memoryOverhead	4096	6144	4096	BZIP2 is highly CPU/memory intensive to decompress. Requires a larger off-heap buffer (6 GB) to prevent YARN SIGKILLs.
spark.hadoop.fs.s3a.input.fadvise	sequential	random	random	Pre-split GZIP is read end-to-end (sequential). BZIP2/CSV are splittable formats requiring block seeking (random).
spark.hadoop.fs.s3a.readahead.range	4194304	4194304	8388608	Kept at 4 MB for compressed formats. Raw CSV uncompressed I/O volume benefits from an increased 8 MB buffer.
spark.sql.files.maxPartitionBytes	134217728	134217728	134217728	Sets target partition size to 128 MB. Naturally splits BZIP2/CSV. Ignored by Spark for non-splittable GZIP.
spark.sql.adaptive.coalescePartitions.initialPartitionNum	4096	4096	4096	Baseline shuffle partitions before AQE dynamically coalesces them down. Matches expected cluster load for 128 MB chunks.
spark.sql.adaptive.advisoryPartitionSizeInBytes	134217728	134217728	134217728	Informs AQE to aim for 128 MB partitions when merging data after a shuffle stage, keeping tasks evenly sized.
spark.executor.cores	8	8	8	8 cores is the optimal sweet spot for parallel HTTP range reads against Ceph RGW across all formats.

Configuration Property	Value (Pre-split GZIP)	Value (BZIP2)	Value (Raw CSV)	Configuration Description & Rationale
spark.executor.memory	28g	28g	28g	28 GB heap remains standard. Streaming reads use internal buffers and do not load the entire file into heap at once.